



Cloud Security Posture Management

Runbook – Remediation

Database instance should not be publicly accessible

Control ID	RDS-003
Control Name	Database instance should not be publicly accessible
Severity	High
Audience	Application Teams · DBA · Cloud Platform · Cloud Security
Tooling	Wiz Cloud Security Platform
Compliance	CSA CCM v4.0.5 → IVS-06 (Network Security)
Version	1.0
Owner	Cloud Security Team
Last Updated	May 2026
Review Cycle	Quarterly

Table of Contents

Table of Contents.....	1
1. Overview	3
2. Description	3
3. Objective	3
4. Triage Process.....	3
4.1 Identify Affected Resources (Wiz)	4
4.2 Categorize Findings	4
5. Manual Mitigation – AWS Console.....	4
5.1 Disable Public Accessibility.....	4
5.2 Review Security Group Rules	5
5.3 Verify Subnet Routing	5
6. Mitigation – AWS CLI.....	5
Step 1 — Check current public accessibility status	5
Step 2 — Disable public accessibility	6
Step 3 — Verify the change	6

7. Mitigation Using Terraform (IaC)	6
7.1 Enforce Private Database Configuration	6
7.2 Recommended Terraform Workflow	7
8. Mitigation Using CloudFormation (IaC)	7
8.1 Enforce Private Access Using CloudFormation	8
8.2 Recommended CloudFormation Workflow	8
9. Document Exception	8
10. Detective Control	9
10.1 Enable AWS Config Rule	9
10.2 Additional Detective Controls	9
11. Validation and Success Criteria	10
12. References	10

1. Overview

Detection Tool	Wiz Cloud Security Platform
Wiz Rule	Cloud Configuration Findings Wiz
Detective Control	AWS Config — rds-instance-public-access-check · Amazon GuardDuty RDS Protection

2. Description

This control identifies RDS and Aurora database instances configured as publicly accessible — meaning they can receive network connections directly from the internet. Publicly accessible databases significantly increase the risk of unauthorized access, brute force attacks, data exfiltration, and exploitation of database engine vulnerabilities.

Databases are intended to reside within private network boundaries and be accessed only through trusted application tiers or controlled administrative paths. Exposing a database directly to the internet violates AWS security best practices and common compliance standards (PCI-DSS, HIPAA, SOC 2).

 **High severity:** A publicly accessible database is an internet-exposed attack surface. Remediate immediately unless a formal exception has been approved.

3. Objective

This runbook provides standardized operational guidance for security, DBA, and cloud operations teams to:

- Validate public accessibility findings in Wiz
- Assess application dependencies before remediation
- Safely disable public accessibility on RDS and Aurora instances
- Handle approved exceptions with documented compensating controls

The goal is to ensure all database instances are deployed in private networks and are not directly accessible from the internet, unless explicitly approved with compensating controls.

4. Triage Process

 **General triage principles** — including how to assess whether a finding is valid, a false positive, or an accepted risk — are defined in the [Master Template – Remediation SOP](#). Complete that review before proceeding with the control-specific steps below.

4.1 Identify Affected Resources (Wiz)

Navigate to Wiz → Cloud Configuration Findings and filter by the rule: "Database instance should not be publicly accessible". Export the list of affected resources.

Collect the following details for each finding:

- Database identifier and instance ARN
- AWS account ID and region
- Database engine and version (RDS MySQL / PostgreSQL / SQL Server, Aurora, etc.)
- PubliclyAccessible flag (true triggers this finding)
- Subnet group and VPC ID
- Security groups attached to the database
- Application or business owner

4.2 Categorize Findings

Category	Applies to	Next Action
A – Remediate Immediately	Database publicly accessible with no documented business need	→ Disable public access. High severity — target same-day action.
B – Validate First	Database publicly accessible with a potential external integration dependency	→ Confirm with application and DBA teams. Validate access paths. Remediate after impact assessment.
C – Exception	Database requires public access for a known, approved integration	→ Proceed to Section 9 (Exception Process). Mandatory compensating controls apply.

 Before remediation: Disabling public access requires a database modification which may cause brief connectivity interruption. Use Apply immediately for non-production and During the next scheduled maintenance window for production.

5. Manual Mitigation – AWS Console

Use this approach for one-off remediations or when the database is not managed by IaC.

5.1 Disable Public Accessibility

1. Navigate to AWS Console → RDS → Databases
2. Select the affected database instance
3. Click Modify
4. Under Connectivity → Additional configuration, set Public access = No
5. Choose Apply immediately (non-production) or During the next scheduled maintenance window (production)
6. Click Continue, then Modify DB Instance to confirm

5.2 Review Security Group Rules

After disabling public access, verify the security group attached to the database does not allow inbound traffic from the public internet:

1. Navigate to EC2 → Security Groups
2. Find the security group attached to the database
3. Review Inbound rules — remove any rule allowing 0.0.0.0/0 or ::/0 on database ports (3306, 5432, 1433, etc.)
4. Replace with specific VPC CIDR ranges or application server security group IDs

Rule	Meaning	Status
Allow 10.0.0.0/16 port 3306	VPC CIDR — private access only	✓ Compliant
Allow app-sg-id port 5432	App tier security group ID	✓ Compliant
Allow 0.0.0.0/0 port 3306	Open to internet	✗ Remove immediately
Allow ::/0 port 5432	Open to internet (IPv6)	✗ Remove immediately

5.3 Verify Subnet Routing

A database can have PubliclyAccessible = false and still be at risk if it is in a subnet with a route to an Internet Gateway. Verify:

- The database subnet group uses private subnets only
- The route table for the DB subnet does NOT contain 0.0.0.0/0 → Internet Gateway
- If a public-facing subnet is in use, move the database to a private subnet group

i Note: VPC Block Public Access is a separate VPC-level feature. It is not a substitute for setting PubliclyAccessible = false at the database instance level. Both controls should be applied.

6. Mitigation – AWS CLI

Run the following commands in sequence. Ensure you have the necessary IAM permissions before proceeding.

Step 1 — Check current public accessibility status

```
aws rds describe-db-instances \
  --db-instance-identifier <DB_IDENTIFIER> \
  --region <REGION> \
```

```
--query
'DBInstances[0].{ID:DBInstanceIdentifier,Public:PubliclyAccessible,Status:DBInstanceStatus}'
\
--output table
```

Step 2 — Disable public accessibility

```
# Non-production - apply immediately
aws rds modify-db-instance \
  --db-instance-identifier <DB_IDENTIFIER> \
  --no-publicly-accessible \
  --apply-immediately \
  --region <REGION>

# Production - apply at next maintenance window (remove --apply-immediately)
aws rds modify-db-instance \
  --db-instance-identifier <DB_IDENTIFIER> \
  --no-publicly-accessible \
  --region <REGION>
```

Step 3 — Verify the change

```
aws rds describe-db-instances \
  --db-instance-identifier <DB_IDENTIFIER> \
  --region <REGION> \
  --query 'DBInstances[0].PubliclyAccessible'
```

7. Mitigation Using Terraform (IaC)

Use this approach when the RDS instance is managed as Infrastructure as Code. Terraform enforces private database configuration by explicitly setting `publicly_accessible = false` and using a private subnet group.

Importing existing databases: If the RDS instance was not originally provisioned via Terraform, import it into state before proceeding. `terraform import aws_db_instance.this <DB_IDENTIFIER>`

7.1 Enforce Private Database Configuration

```
resource "aws_db_instance" "app_db" {
  identifier          = "app-database"
  engine              = "mysql"
  engine_version      = "8.0"
  instance_class      = "db.t3.medium"
  allocated_storage   = 20

  # Explicitly disable public access
  publicly_accessible = false

  # Must be a private subnet group
  db_subnet_group_name = aws_db_subnet_group.private.name

  # Restrict access to app tier security group
  vpc_security_group_ids = [aws_security_group.app_db_sg.id]
}

resource "aws_db_subnet_group" "private" {
  name          = "private-db-subnets"
  subnet_ids    = [aws_subnet.private_a.id, aws_subnet.private_b.id]
}
```

7.2 Recommended Terraform Workflow

1. Wiz identifies the publicly accessible database
2. Security and DBA teams validate business dependency and confirm maintenance window
3. If not in Terraform state, run terraform import
4. Set publicly_accessible = false and confirm db_subnet_group_name references a private subnet group
5. Run terraform plan — review the proposed changes carefully
6. Obtain peer review and approval of the plan output
7. Run terraform apply — schedule during maintenance window for production
8. Verify in Wiz — finding should resolve within the next scan cycle

8. Mitigation Using CloudFormation (IaC)

Use this approach when the RDS instance is managed via CloudFormation. Set PubliclyAccessible: false and reference a private subnet group.

Drift detection: If the database was modified manually outside of CloudFormation, the stack's view will be stale. Run drift detection before executing a change set. AWS Console → CloudFormation → Stack → Stack actions → Detect stack drift

8.1 Enforce Private Access Using CloudFormation

```
DatabaseInstance:
  Type: AWS::RDS::DBInstance
  Properties:
    DBInstanceIdentifier: app-database
    DBInstanceClass: db.t3.medium
    Engine: mysql
    EngineVersion: '8.0'
    PubliclyAccessible: false # Explicitly disabled
    DBSubnetGroupName: !Ref PrivateDBSubnetGroup
    VPCSecurityGroups:
      - !Ref AppDBSecurityGroup

PrivateDBSubnetGroup:
  Type: AWS::RDS::DBSubnetGroup
  Properties:
    DBSubnetGroupDescription: Private subnets for RDS
    SubnetIds:
      - !Ref PrivateSubnetA
      - !Ref PrivateSubnetB
```

8.2 Recommended CloudFormation Workflow

1. Run drift detection on the stack before making any changes
2. Security and DBA teams validate business and operational impact
3. Update template to set `PubliclyAccessible: false` and reference a private subnet group
4. Create a change set and review the proposed changes carefully
5. Obtain peer review and approval of the change set
6. Execute the stack update — schedule during maintenance window for production

9. Document Exception

Use the exception process when a database genuinely requires public accessibility for an approved external integration (Category C from Section 4.2).

Exception Process: Submit and manage exceptions using the formal exception governance process. All required fields, approval workflows, and expiration timelines are defined there. → [Wiz CSPM Exception Governance](#)

At minimum, every exception must include:

- Business justification for why public access is required
- Approver name and approval date
- Exception expiration date (maximum 90 days — shorter than standard due to High severity)
- Compensating controls in place (e.g., IP allow-listing, TLS enforced, CloudTrail and GuardDuty monitoring, no 0.0.0.0/0 security group rules)

10. Detective Control

This section describes how publicly accessible databases are detected and how to independently verify findings using AWS-native tooling.

AWS does not prevent databases from being created as publicly accessible by default. Wiz continuously evaluates database network settings and generates findings where public exposure is detected.

10.1 Enable AWS Config Rule

1. Navigate to AWS Config → Rules → Add Rule
2. Search for rds-instance-public-access-check (AWS managed rule)
3. Save and enable the rule

Property	Value
AWS Config Rule	rds-instance-public-access-check
Detection scope	All RDS and Aurora instances in the account
Result	Continuous compliance visibility · audit-ready evidence · independent of Wiz scan cycle

10.2 Additional Detective Controls

Consider enabling the following supplementary controls:

- Amazon GuardDuty — RDS Protection detects suspicious database connection attempts from unusual sources
- VPC Flow Logs — captures inbound connection attempts to database subnets for forensic analysis
- RDS Enhanced Monitoring — monitors for unusual connection patterns in real time

11. Validation and Success Criteria

After completing remediation, verify success using the following checklist before closing the finding.

☐	Re-check the specific resource in Wiz and confirm the finding status is Resolved or No longer applicable.
☐	RDS console shows PubliclyAccessible = No for the affected instance.
☐	AWS CLI describe-db-instances confirms publicly_accessible = false.
☐	Security group has no inbound rules allowing 0.0.0.0/0 or ::/0 on database ports.
☐	Database subnet route table has no route to an Internet Gateway.
☐	Application connectivity confirmed — no service disruption after the change.
☐	AWS Config rule rds-instance-public-access-check shows Compliant.
☐	Update the Finding Management Tool with the remediation outcome and date.
☐	If an exception was filed (Category C), confirm it is logged with an expiration date.
☐	Update runbook or IaC documentation if any environment-specific steps were required.

12. References

- [Wiz Cloud Configuration Rules](#)
- [Amazon GuardDuty RDS Protection](#)
- [AWS Config Rule — rds-instance-public-access-check](#)
- [AWS RDS — Modifying a DB Instance](#)
- [CSA CCM v4.0.5 Cargill Framework \(Wiz\)](#)
- [Cloud Security Navigator \(CSN\)](#)
- [Master Template – Remediation SOP](#)
- [Wiz CSPM Exception Governance](#)
- [CSPM Finding Management Tool](#)